# SESSION 10 — Merging & Joining DataFrames

## Why Merging is Important

In real-world scenarios, data is often stored in **multiple tables/files**.

Examples:

Customer Table:

| Customer_ID | Name |
|---|---|
| 1 | Aman |
| 2 | Priya |

Sales Table:

| Customer_ID | Amount |
|---|---|
| 1 | 500 |
| 2 | 700 |

To analyze data properly, we need to **combine these tables**.

---

# Solution

Pandas provides:

merge()

---

# Key Concept

Merge is similar to **SQL JOIN**

# merge() Function

## Syntax

```
pd.merge(left_df, right_df, how="type", on="column")
```

---

## Parameters

left_df → first DataFrame
right_df → second DataFrame

how → type of join:

• "inner" → common values only
• "left" → all values from left
• "right" → all values from right
• "outer" → all values from both

on → column name used for joining

---

# Use Case

• Combine customer data with sales
• Combine datasets from multiple sources

```
import pandas as pd

customers = pd.DataFrame({
    "Customer_ID": [1,2,3],
    "Name": ["Aman","Priya","Rahul"]
})

sales = pd.DataFrame({
    "Customer_ID": [1,2,4],
    "Amount": [500,700,300]
})

customers, sales
```

```
(   Customer_ID   Name
 0            1   Aman
 1            2  Priya
 2            3  Rahul,
    Customer_ID  Amount
 0            1     500
 1            2     700
 2            4     300)
```

# 1. Inner Join

Keeps only matching values in both tables.

```
pd.merge(customers, sales, how="inner", on="Customer_ID")
```

| | Customer_ID | Name | Amount |
|---|---|---|---|
| **0** | 1 | Aman | 500 |
| **1** | 2 | Priya | 700 |

## 2. Left Join

Keeps all rows from left table.

```
pd.merge(customers, sales, how="left", on="Customer_ID")
```

| | Customer_ID | Name | Amount |
|---|---|---|---|
| **0** | 1 | Aman | 500.0 |
| **1** | 2 | Priya | 700.0 |
| **2** | 3 | Rahul | NaN |

## 3. Right Join

Keeps all rows from right table.

```
pd.merge(customers, sales, how="right", on="Customer_ID")
```

| | Customer_ID | Name | Amount |
|---|---|---|---|
| **0** | 1 | Aman | 500 |
| **1** | 2 | Priya | 700 |
| **2** | 4 | NaN | 300 |

## 4. Outer Join

Keeps all rows from both tables.

```
pd.merge(customers, sales, how="outer", on="Customer_ID")
```

| | Customer_ID | Name | Amount |
|---|---|---|---|
| **0** | 1 | Aman | 500.0 |
| **1** | 2 | Priya | 700.0 |
| **2** | 3 | Rahul | NaN |
| **3** | 4 | NaN | 300.0 |

# Merge on Multiple Columns

## Syntax

pd.merge(df1, df2, on=["col1","col2"])

---

## Use Case

When one column is not enough to uniquely identify data.

```
df1 = pd.DataFrame({
    "ID":[1,1,2],
    "City":["Delhi","Mumbai","Delhi"],
    "Sales":[100,200,300]
})

df2 = pd.DataFrame({
    "ID":[1,2],
    "City":["Delhi","Delhi"],
    "Target":[150,350]
})

pd.merge(df1, df2, on=["ID","City"])
```

| | ID | City | Sales | Target |
|---|---|---|---|---|
| **0** | 1 | Delhi | 100 | 150 |
| **1** | 2 | Delhi | 300 | 350 |

# Default Behavior of pd.merge()

When you do:

pd.merge(df1, df2, on=["ID","City"])

and do NOT specify `how`

---

## Default Join Type

The default join is:

INNER JOIN

---

## What is INNER JOIN?

It keeps only those rows where:

- Values match in BOTH DataFrames
- Based on the given key(s)

---

## Important Rule

If no match exists → row is dropped

---

## Syntax Reminder

pd.merge(df1, df2, how="inner", on=["ID","City"])

is SAME as:

pd.merge(df1, df2, on=["ID","City"])

## Demo — Customer + Sales Merge

Goal:

Combine customer details with their sales data.

```
merged_df = pd.merge(customers, sales, how="left", on="Customer_ID")

merged_df
```

| | Customer_ID | Name | Amount |
|---|---|---|---|
| **0** | 1 | Aman | 500.0 |
| **1** | 2 | Priya | 700.0 |
| **2** | 3 | Rahul | NaN |

## Question 1 — Default Merge Trap

Create two DataFrames:

df1:

| ID | Name |
|---|---|
| 1 | Aman |
| 2 | Priya |
| 3 | Rahul |

df2:

| ID | Score |
|---|---|
| 2 | 80 |
| 3 | 90 |
| 4 | 70 |

Tasks:

1 Perform merge WITHOUT specifying `how`
2 Explain which rows will appear and why
3 How many rows will be in output?

---

## Question 2 — Duplicate Key Explosion

Create:

df1:

| ID | Product |
|---|---|
| 1 | A |
| 1 | B |

df2:

| ID | Price |
|---|---|
| 1 | 100 |
| 1 | 200 |

Tasks:

1 Perform inner merge
2 Count number of rows in output
3 Explain why rows increase

(Hint: Cartesian multiplication)

---

## Question 3 — Left Join with Missing Values

Create:

df1:

| ID | Name |
|---|---|
| 1 | Aman |
| 2 | Priya |
| 3 | Rahul |

df2:

| ID | Salary |
|---|---|
| 1 | 50000 |

Tasks:

1 Perform left join
2 Identify rows with missing Salary
3 Fill missing Salary with 0

---

## Question 4 — Multiple Key Merge Trap

Create:

df1:

| ID | City | Sales |
|---|---|---|
| 1 | Delhi | 100 |
| 1 | Mumbai | 200 |

df2:

| ID | City | Target |
|---|---|---|
| 1 | Delhi | 150 |

Tasks:

1 Merge using only ID
2 Merge using ID + City

3 Compare both outputs

Explain:

Why results are different?

---

## Question 5 — Real-World Merge Scenario

Create:

customers:

| Customer_ID | Name |
|---|---|
| 1 | Aman |
| 2 | Priya |
| 3 | Rahul |

orders:

| Customer_ID | Order_Amount |
|---|---|
| 1 | 500 |
| 1 | 700 |
| 2 | 300 |

Tasks:

1 Merge both datasets
2 Calculate total order amount per customer
3 Identify customers with no orders
4 Replace missing values with 0

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```